Visit Planner: Prompt Documentation and Video Script

Project: Visit Planner, a healthcare clinic appointment assistant (website with a Java or Python backend) AI tool used: Claude (Anthropic)

Note: edit the wording below so it matches what you personally typed. The prompts here follow the real flow of this project.

1. Project summary

Visit Planner helps a patient describe why they need a clinic visit. It estimates urgency (Emergency, Same day, Within 48 hours, Routine), suggests an appointment type, explains its reasoning, estimates no-show risk, and drafts a reminder message. A Flask and SQLite backend adds booking and a staff queue sorted by urgency. All data is fictional and the app states that it is not medical advice.

2. Prompt log

Prompt 1: Understand the brief

Prompt: I pasted the full assessment brief and asked Claude to help me complete it.

Why: I wanted the AI to read the whole task first, so the requirements (video, prompt document, project link, healthcare or technology domain) shaped everything after.

Result: Claude summarised the requirements, flagged that the web-scraping option (bulk collection of GP contact details) carries privacy and legal risk, and proposed a clinic appointment assistant.

Prompt 2: Choose the track

Prompt: "website"

Why: I picked Website Development because it is easy to demonstrate in a 2-minute video.

Result: A working single-page app with symptom input, red-flag checkboxes, urgency result, no-show risk and a draft reminder.

Prompt 3: Expand to full stack

Prompt: "Can you build an entire website based on technology or healthcare, your choice, frontend and backend or either?"

Why: The first version was frontend only. I wanted to show I can combine a frontend with an API and a database.

Result: Flask backend with POST /api/triage, POST /api/bookings and GET /api/staff/queue, plus a README and requirements file.

3. Prompt revisions to add (examples of iterative improvement)

Use these as follow-up prompts, then record what changed after each one.

1. **Safety:** "Add an emergency override so any red-flag checkbox forces the Emergency level and hides booking." Reason: a health tool must never route an emergency into a normal queue.
2. **Transparency:** "Show a plain-language list of reasons behind every result." Reason: rule-based output is easier to trust and to audit than a hidden score.
3. **Accessibility:** "Make the page work in dark mode, on mobile, with visible keyboard focus and screen reader labels."
4. **Backend wiring:** "Replace the frontend triage function with a fetch call to /api/triage and add a booking form that calls /api/bookings."
5. **Hardening:** "Validate input lengths, reject empty requests, and add authentication to the staff queue."
6. **Testing:** "Write test cases for emergency wording, children and seniors, long lead times, and repeat no-shows."

4. Design decisions and challenges

- **Rule-based triage instead of a live AI call.** The AI helped design and write the rules, which keeps results explainable and predictable. A hosted page also cannot call an AI API directly.
- **Safety first.** Emergency wording and red flags override everything else, and the app tells the person to call emergency services.
- **Privacy.** Only fictional sample data is used, and I avoided the scraping option for privacy reasons.

- **Known limits.** Keyword matching is easy to fool, the staff queue has no login yet, and this is a prototype, not a medical device.

5. Video script (about 2.5 minutes)

0:00 to 0:20, Introduction. "Hi, I'm [your name]. For this task I built Visit Planner, a healthcare appointment assistant. A patient describes why they need a visit, and it suggests how urgent it is, what kind of appointment to book, and drafts a reminder to reduce no-shows."

0:20 to 0:45, AI tools. "I used Claude as my main AI tool, for reading the brief, designing the logic, and writing the frontend and backend code. I chose it because it handles long instructions well and explains its reasoning, which helped me review the code."

0:45 to 1:15, Prompt engineering. "I started by giving the full brief and asking for a plan. Then I chose the website track and asked for a full-stack version. I refined with follow-ups: an emergency override, reasons shown for every result, accessibility, and input validation. Being specific about one change per prompt gave better results than asking for everything at once." [Show the prompt document on screen.]

1:15 to 1:35, Approach and challenges. "My main concern was safety. A health tool must not downplay an emergency, so red flags override everything. I also chose transparent rules instead of a black box, and I avoided collecting real patient or doctor data."

1:35 to 2:15, Demo. [Screen share.] "Here I type 'sore throat and fever since yesterday'. It returns a Within 48 hours level, an acute illness visit, the reasons, a no-show risk, and a reminder message I can copy. Now I tick 'chest pain'. It switches to Emergency and tells me to call emergency services. On the backend, this POST request creates a booking, and the staff queue shows it sorted by urgency."

2:15 to 2:30, Wrap-up. "Next steps are staff login, a real calendar, and clinical review of the rules. Thanks for watching. The code and prompt document are linked in my email."

6. Submission checklist

- [ ] Record the video (minimum 2 minutes) and upload it to Google Drive with access set to anyone with the link
- [ ] Export this document as PDF or Word
- [ ] Put the project folder on GitHub and make it public
- [ ] Email jobs@gokulinfocare.com with the video, prompt document and project link

- [ ] Test every link in a private browser window before sending